Report

CS5013: Assignment 1

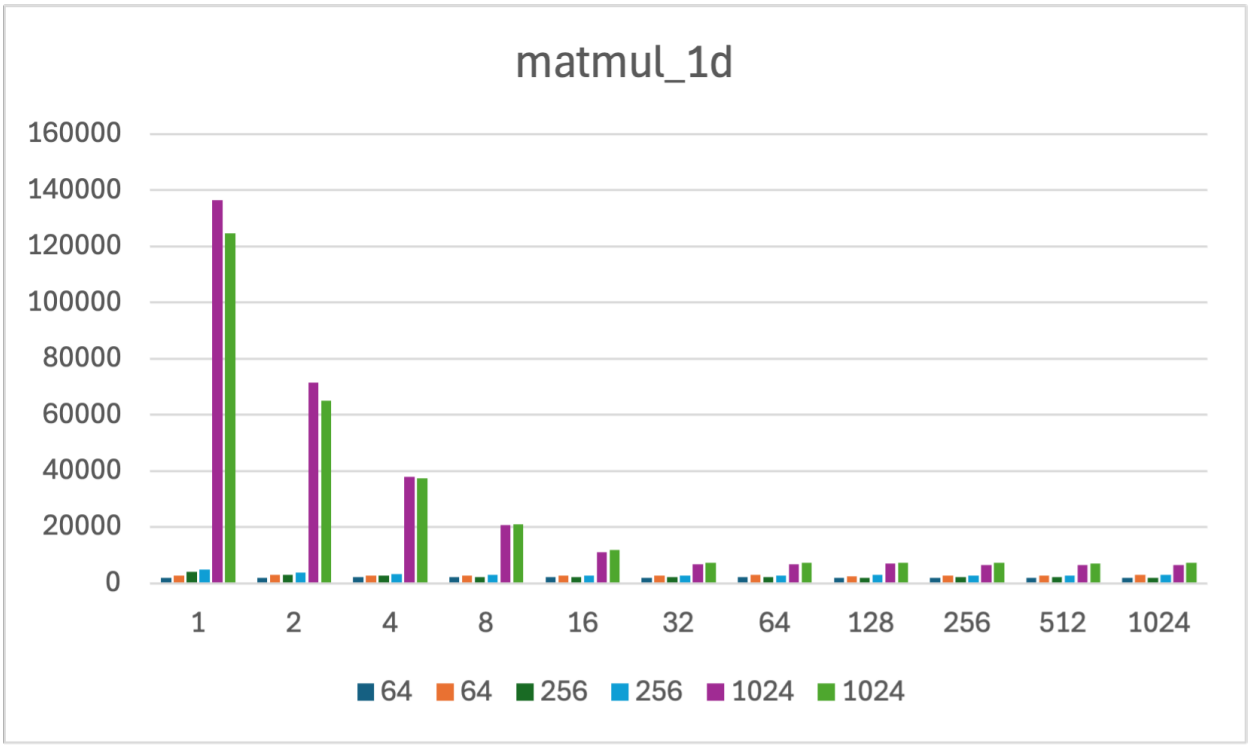
Getting Started with Matrices !!

Problem 1.1

Table

Matrix Size	Config (M, N)	Time (Run 1)	Time (Run 2)
64	4096, 1	2036.8	2812.6
64	2048, 2	1987	2888.4
64	1024, 4	2145.8	2744.4
64	512, 8	2151	2687.8
64	256, 16	2102.8	2657.8
64	128, 32	2001.8	2744.6
64	64, 64	2097.2	2950
64	32, 128	1966.8	2532
64	16, 256	1995.8	2709
64	8, 512	1981	2753.2
64	4, 1024	1980.4	2902
256	65536, 1	4085.4	4837.2
256	32768, 2	2932.6	3741.6
256	16384, 4	2602.4	3318
256	8192, 8	2116.4	2979.4
256	4096, 16	2138.6	2815.8
256	2048, 32	2151.8	2734.8
256	1024, 64	2117	2845.2
256	512, 128	2028.8	2871.6
256	256, 256	2054.4	2766.8
256	128, 512	2160.2	2859.2
256	64, 1024	1994.4	2901
1024	1048576, 1	136466.8	124568.6
1024	524288, 2	71397.2	65111.6
1024	262144, 4	37825	37378.8
1024	131072, 8	20710	21057.4
1024	65536, 16	11181.6	11882.2
1024	32768, 32	6734.2	7243.6
1024	16384, 64	6728.4	7364
1024	8192, 128	6979.2	7423
1024	4096, 256	6496.8	7162.2
1024	2048, 512	6581.8	7146.2
1024	1024, 1024	6535.8	7228.4

Graph



=

Analysis

Here we don't see any scaling with number of threads per thread block up until we reach matrices of size around 1024 X 1024. As the number of threads per block increases the multiplication time decreases with it. It follows an exponential curve sort of. As we go from 1 to 2 the total time gets slashed by around 1/2 and similarly from 2 to 4 and 4 to 8 (In the graph the horizontal axes has labels 1 through 11 but it is in reality powers of 2 from 1 to 1024 (actual_label $= 2^{\text{displayed\_label}-1}$)). There are some outliers here that have randomly an extremely high runtime as compared to their counterparts. Another major thing to look at is that the second run of all size matrices has consistently higher runtime than the first run. The overhead incurred here is quite high (around 50% - 100% of runtime) and can most probably be attributed to environment factors. One thing to note is that this scaling goes on only till 32 threads after which we don't see any improvement. This is due to the reason that a warp contains only 32 threads. What actually happens is in the case of smaller matrices the number of thread blocks generated isn't that high, so the number of

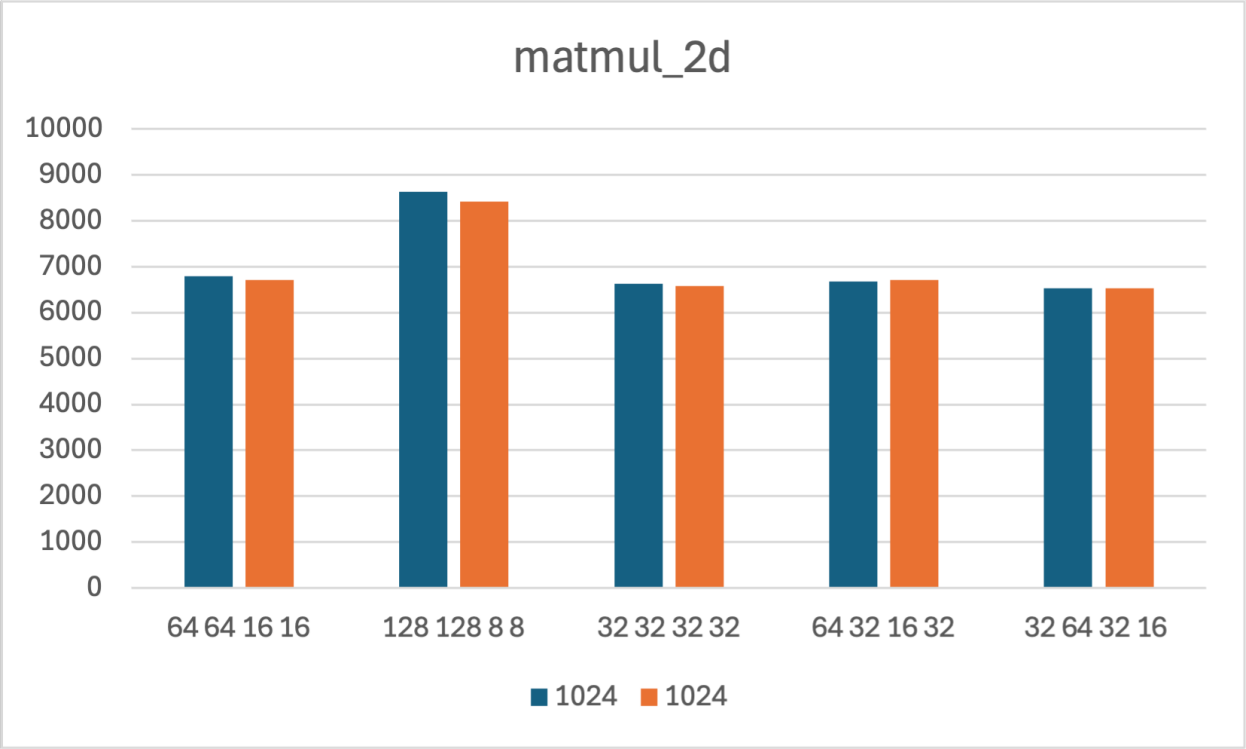
warps created is also not that high (in the case when number of threads in a thread block is $\leq$ 32 the number of warps is equal to number of thread blocks). So the number of available streaming multiprocessors are able to run all the generated warps. But in case of 1024 X 1024 the number of warps generated is so high that we are not able to run all at the same time. So when we increase from 1 through 32 we actually increase the number of threads running in parallel. After 32 the extra threads get put into new warps so this advantage is lost. So after 32 we don't see any increase in performance.

Problem 1.2

Table

Matrix Size	gridSize, blockSize	Time (Run 1)	Time (Run 2)
64	8X8, 8X8	2039.6	2049.2
64	4X4, 16X16	1811.4	2254.4
64	2X2, 32X32	1846.2	2209.8
64	8X4, 8X16	1729.6	2106.8
64	4X8, 16X8	1744.6	2090.8
256	16X16, 16X16	1874.6	2255.2
256	32X32, 8X8	1929.6	2186.8
256	8X8, 32X32	1936.8	2335.8
256	16X8, 16X32	2232.2	2268.6
256	8X16, 32X16	2344.4	2254.6
1024	64X64, 16X16	6792.2	6701.4
1024	128X128, 8X8	8625.6	8421.2
1024	32X32, 32X32	6622	6570
1024	64X32, 16X32	6674.2	6712.6
1024	32X64, 32X16	6535.6	6536.4

Graph



Here only the data for size 1024 X 1024 was plotted as it was the only one with a point of interest.

Analysis

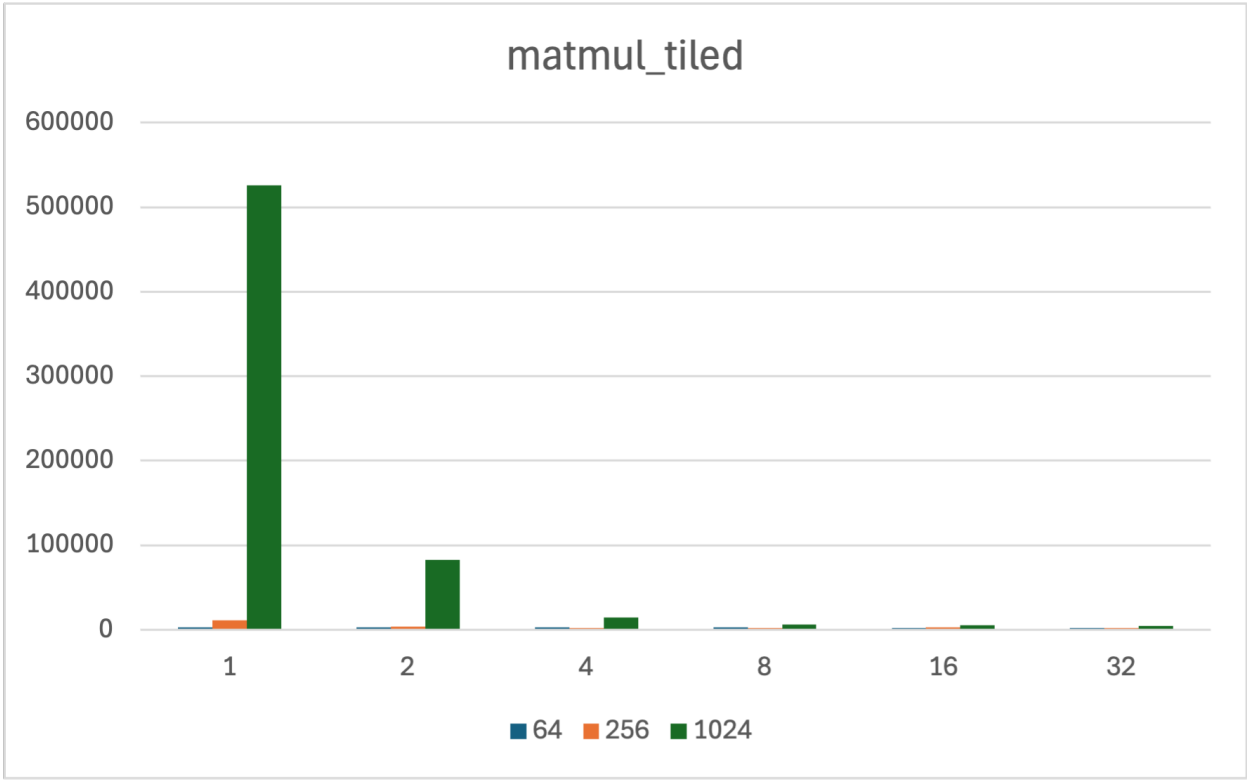
In the configuration where the block size is 8 X 8 we see a higher runtime than the rest in both the runs. The reason is most probably the way memory access is taking place in the running of the algorithm. The 8 X 8 blocks are not utilizing the cache line as effectively as 16 X 16. Assuming cache line size of 128 bytes so it means 32 values. So 8 X 8 uses 25% only while 16 X 16 uses 50%. So maybe the 50% use is enough that latency hiding occurs and we don't see a drop in throughput. But this is mere speculation as there is no evidence to back this up. There is the possibility of better memory coalescing as well because due to experiments run (results not reported) it can be concluded that the x dimension of a thread block has the effect of changing the throughput. The under utilization possibility of the warp is no more as we have 64 threads in a block this time. The rest of the values are similar to matmul_1d values.

Problem 2

Table

Matrix Size	Tile Size	Global Access	Shared Access	Time
64	1	528384	1048576	2406
64	2	266240	786432	2358.6
64	4	135168	655360	2390.4
64	8	69632	589824	2473.6
64	16	36864	557056	2178.6
64	32	20480	540672	2269
256	1	33619968	67108864	10963.8
256	2	16842752	50331648	3376.8
256	4	8454144	41943040	2308.8
256	8	4259840	37748736	2304.4
256	16	2162688	35651584	2481.6
256	32	1114112	34603008	2319.8
1024	1	2148532224	4294967296	525621.4
1024	2	1074790400	3221225472	82416.8
1024	4	537919488	2684354560	14427.4
1024	8	269484032	2415919104	5947
1024	16	135266304	2281701376	4880.4
1024	32	68157440	2214592512	4292.4

Graph



Analysis

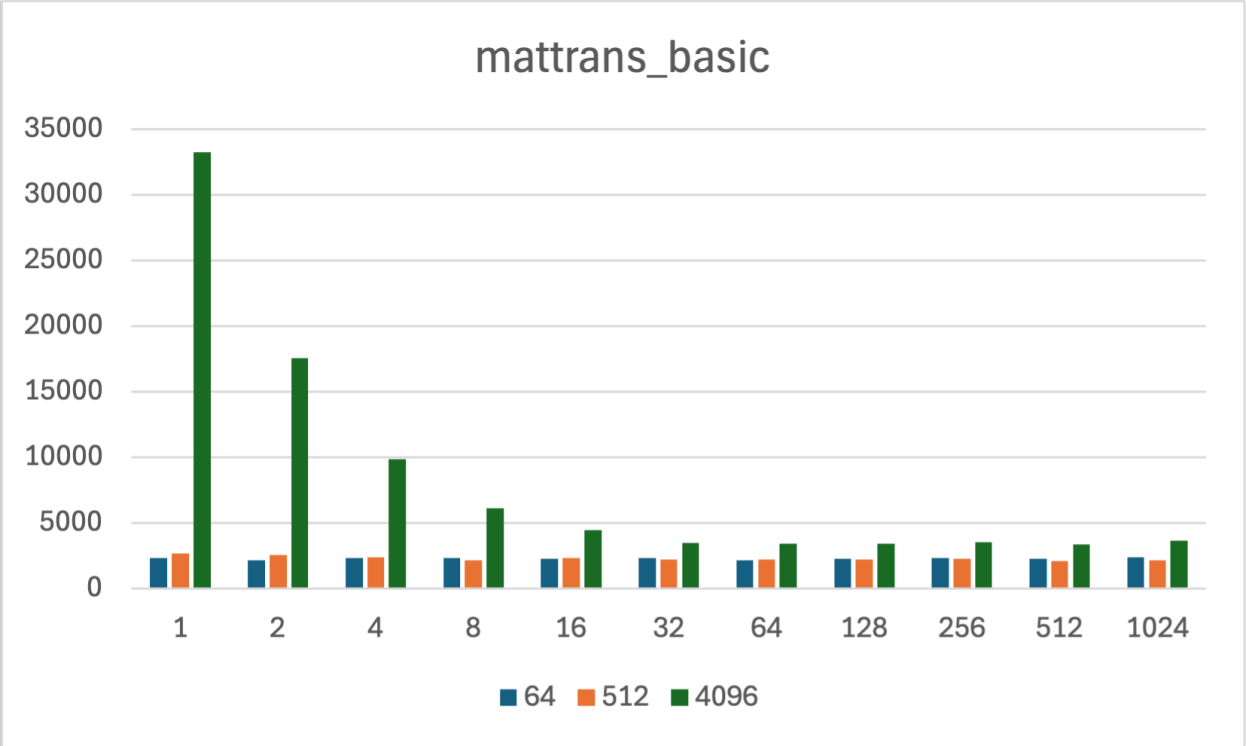
The first thing to note is that the runtime for smaller tiles has almost exploded from around 6000 we have gone to 500000 an approx 80x increase and similar cases for smaller sizes as well. This majorly due to several overheads that are incurred in using shared memory from algorithm complexity increase to use of __syncthreads() all give this increased overhead. But as tile size starts to increase we see similar exponential drop in runtime. In the case of 1024 it surpasses the matmul_1d and matmul_2d by a huge margin, signifying the gains we get from larger matrices. Here the fastest tile size in all the cases is 32 X 32 as it has the highest reuse compared to others. This is the main goal of tiling algorithm in matrix multiplication to maximize cache reuse. We use tiling in normal scenario so that before a cache line is expelled we reuse it as much as possible. It's like trying to make the matrix small enough so that it fits in the cache. Similar concept is applied here. We are essentially trying to fit the matrix in the SM. So obviously the bigger the chunk we fit the fewer global accesses happen and faster the algorithm runs.

Problem 3

Table

Matrix Size	Config (M, N)	Time (Run 1)
64	4096, 1	2292.6
64	2048, 2	2136.2
64	1024, 4	2307.4
64	512, 8	2314.6
64	256, 16	2254.4
64	128, 32	2331.6
64	64, 64	2132.8
64	32, 128	2273
64	16, 256	2311.6
64	8, 512	2268
64	4, 1024	2349.4
512	262144, 1	2654
512	131072, 2	2550.8
512	65536, 4, 4	2339.6
512	32768, 8	2129.8
512	16384, 16	2295.8
512	8192, 32	2174.8
512	4096, 64	2216.2
512	2048, 128	2209
512	1024, 256	2278.2
512	512, 512	2078.2
512	256, 1024	2162.8
4096	16777216, 1	33193.2
4096	8388608, 2	17549.4
4096	4194304, 4	9859.6
4096	2097152, 8	6108
4096	1048576, 16	4436
4096	524288, 32	3455
4096	262144, 64	3397.4
4096	131072, 128	3398
4096	65536, 256	3520.8
4096	32768, 512	3341.4
4096	16384, 1024	3628.4

Graph



Analysis

Here again we see the same phenomenon observed in **Problem 1.1**. That is under utilization of warps. Other than that nothing unusual happens here. Everything is explained by that.

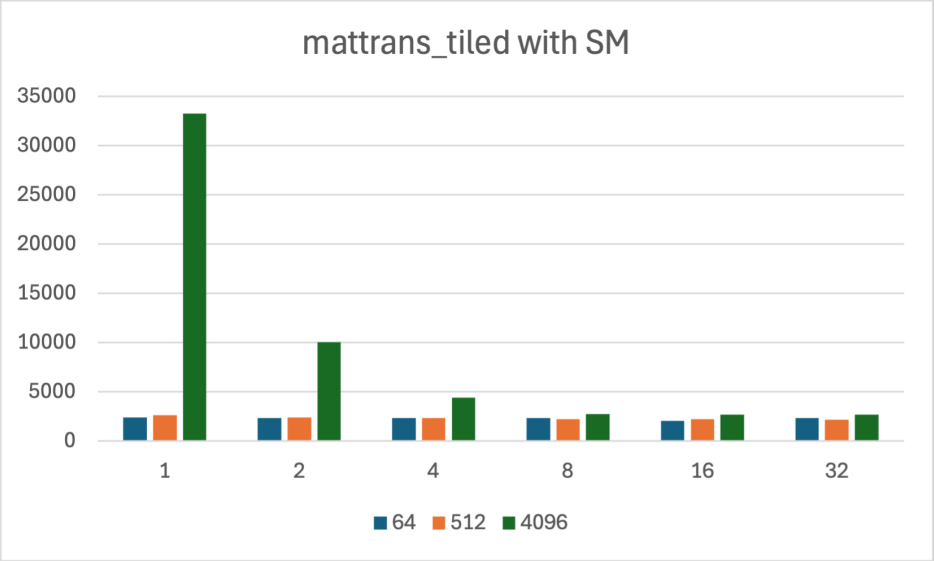
Problem 4

Table

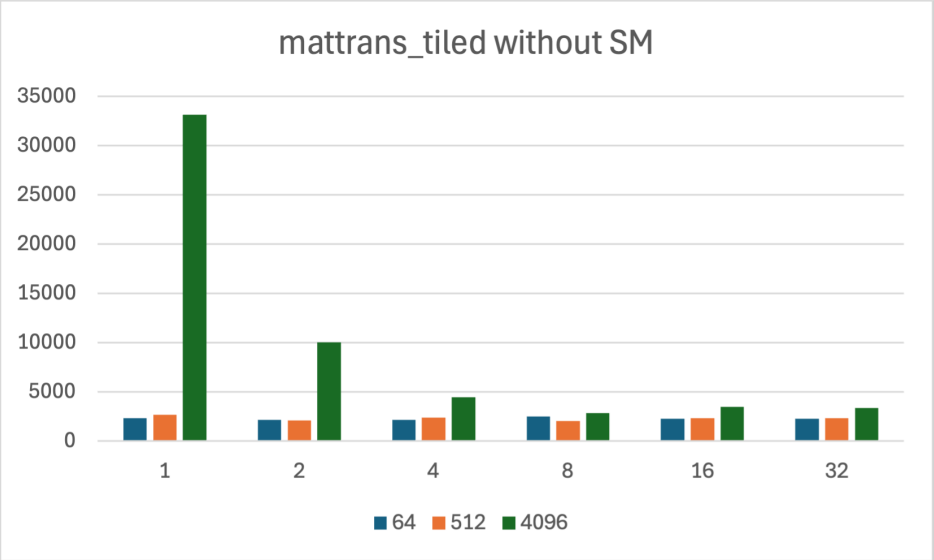
Matrix Size	Tile Size	Time (With SM)	Time (Without SM)
64	1	2382.4	2351.4
64	2	2362.6	2191.4
64	4	2346.2	2155.2
64	8	2371.2	2513.6
64	16	2065.8	2261
64	32	2350.6	2294.2
512	1	2658.2	2698.2
512	2	2398.8	2095.8
512	4	2340.8	2392.2
512	8	2205.4	2062.2
512	16	2234.6	2340.2
512	32	2180	2346.2
4096	1	33249.6	33128.2
4096	2	10057.4	10026
4096	4	4407.2	4484
4096	8	2740.8	2871.2
4096	16	2707.4	3488.4
4096	32	2678.4	3359.4

Graph

With SM



Without SM



Analysis

Here again we see a combination of factors coming together. The first and foremost thing to note is that using shared memory has no significant impact on runtime even

in large matrix sizes like 4096 X 4096. The major performance gain one sees is due to the same phenomenon as **Problem 1.1**. This is why the runtime doesn't decrease after 8, as $4 * 4 = 16$ but $8 * 8 = 64$ so at 8 all the warps are fully utilized. The plateauing of performance after 8 is a major indicator of this. Shared Memory doesn't have an impact here because taking transpose is a simple operation and each thread accesses data only once and each data is accessed only once. so there is no scope of reuse as such. So SM has no gains here it only complicates things by adding overhead.